

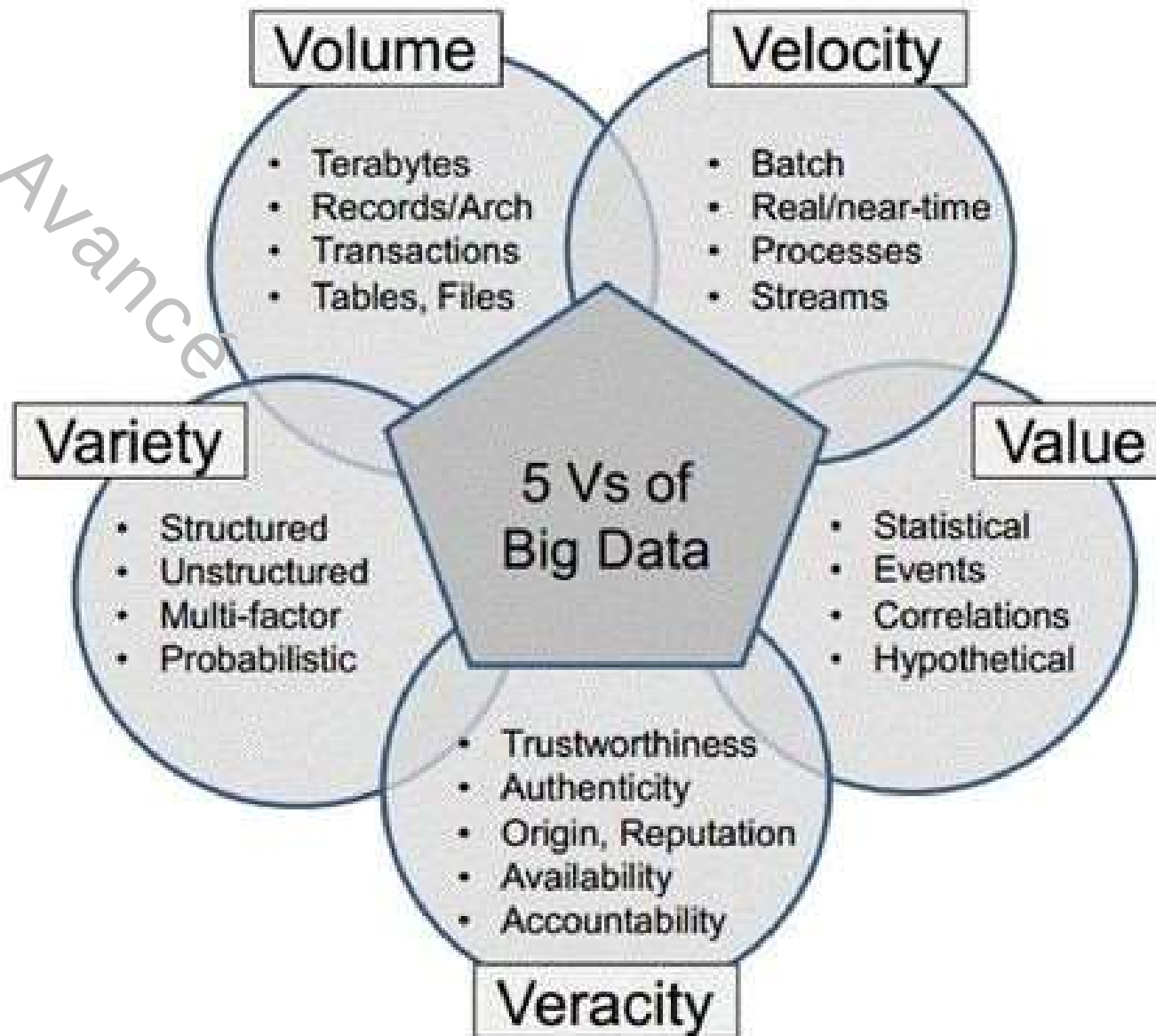
CH HDFS

- ☐ Big Data: Introduction
- ☐ Hadoop: presentation, use case, integration, ecosystem
- ☐ HDFS
 - Architecture
 - Reading/Writing
 - Rack Awareness
 - Federation
 - Commands
 - Exercise

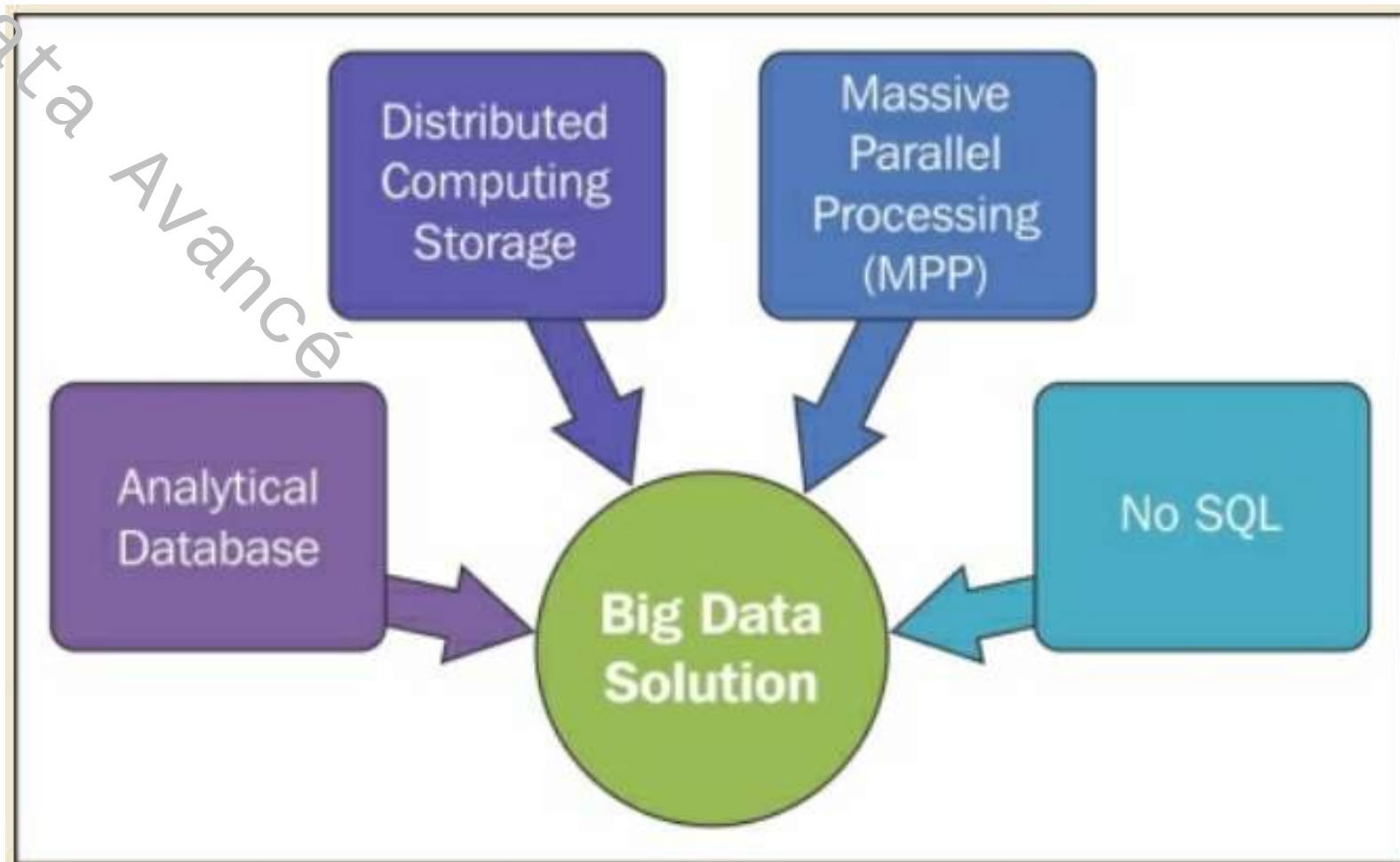


- Big Data systems have emerged due to some problems and limitations in traditional systems.
 - Traditional systems are good for processing (Online Transaction Processing (OLTP) and Business Intelligence (BI), but they are not easily scalable (scale) given
 - * costs,
 - * efforts
 - * management
 - * heavy calculations
 - * **memory problem**
- Traditional systems are **severely lacking** in the **analysis of data** (scientific data, ...)
- **Examples of big data use cases** are predictive analytics, fraud analysis, machine learning, trend identification, data analysis, semi-structured and unstructured data processing and analysis.

Problem related to Big Data



- Big Data systems is a terminology that refers to the challenges that arise due to the exponential growth of data (The 5Vs):
 - Capture
 - Storage
 - Searching
 - Sharing
 - Analysis
 - Visualisation
- Technologies that can solve Big Data problems should use the following architectural strategy:
 - Distributed computing system
 - Massively Parallel Processing (MPP)
 - NoSQL (not just SQL)



- Big Data use case models: there are many technological scenarios

- **Big Data as a storage model**

- Big Data systems can be used as a storage model or as a **data warehouse**, where data from multiple sources, even with different data types, can be stored and used later.

- **Big Data as a data transformation model**

- Big Data systems can be designed to perform transformation (data loading & cleansing). Many transformations can be carried out more quickly than traditional systems because of the parallelism.

- Transformation is a phase in the **ETL (Extraction Transformation Loading) process**.

- **Big Data for a data analysis model**

Data analysis is of more general interest for Big Data systems, where huge amounts of data can be analyzed to produce statistical reports and data insights

- **Big Data for real-time data**

Big Data systems incorporate certain libraries and systems that are capable of processing large-scale real-time data. Real-time processing for a large and complex requirement involves many challenges such as performance, scalability, availability, resource management, low latency, etc.

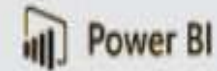
- **Big Data for a low-latency cache model**

Big Data systems can be tuned as a special case for a low latency system, where reads are much higher and updates are low, which can retrieve data faster and can be stored in memory, which can further improve performance.

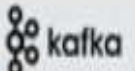
Introduction



Visualization & Analytics



Compute



Storage



Distributions & Data Warehouse

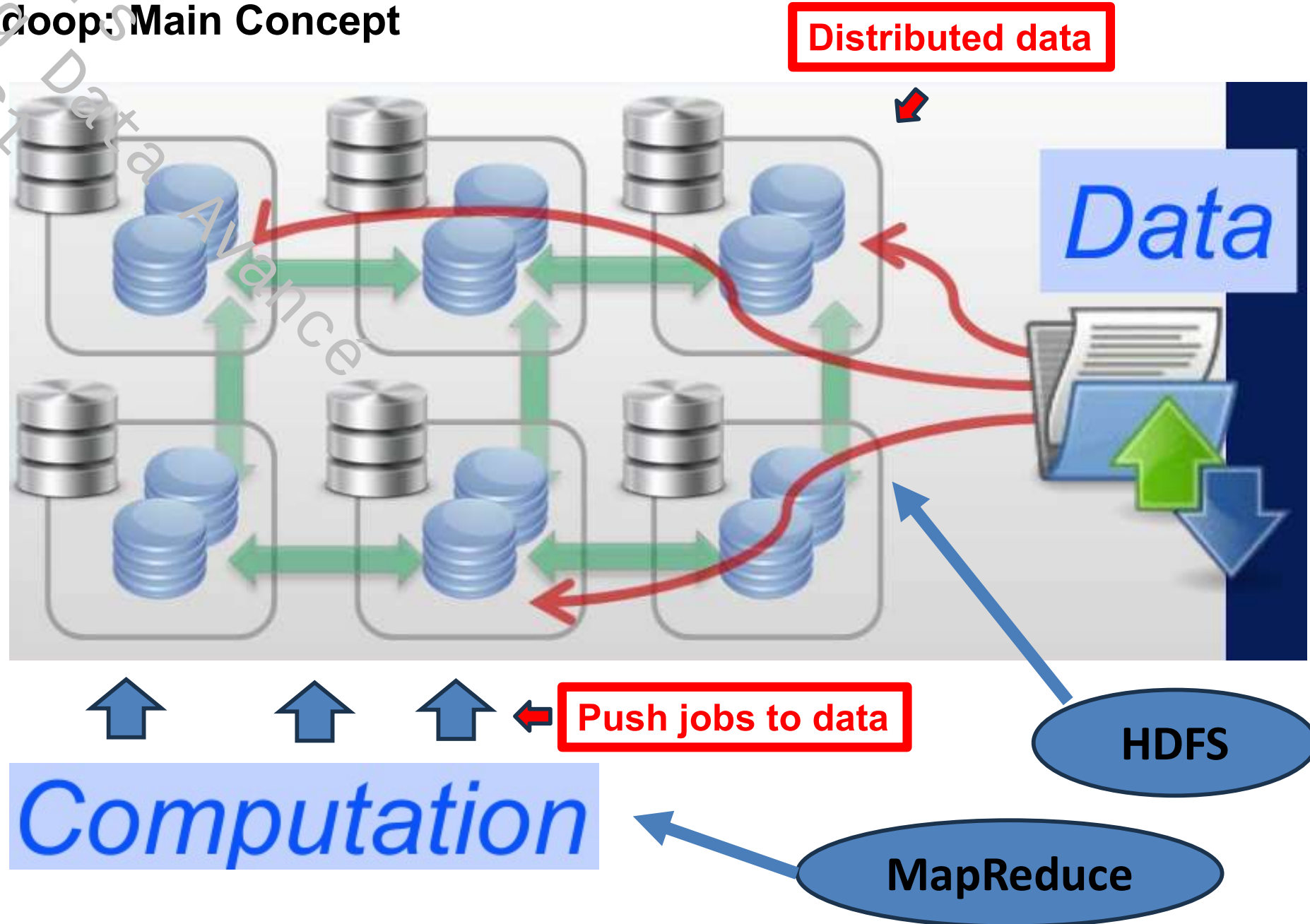


- In the field of Big Data, the most widely used system is **Hadoop**. Hadoop is an **open source implementation of Big Data**, which is widely accepted in the industry.
- Hadoop's performance is impressive and, in some cases, incomparable to other systems.
- Hadoop is used in industry for **large-scale, massively parallel and distributed** data processing.
- Hadoop is highly fault-tolerant and configurable at several levels
- In Hadoop, distributed computing is managed by **HDFS**, and parallel processing is managed by **Mapreduce**.
- In short, Hadoop is a combination of HDFS and Mapreduce.

Hadoop: introduction

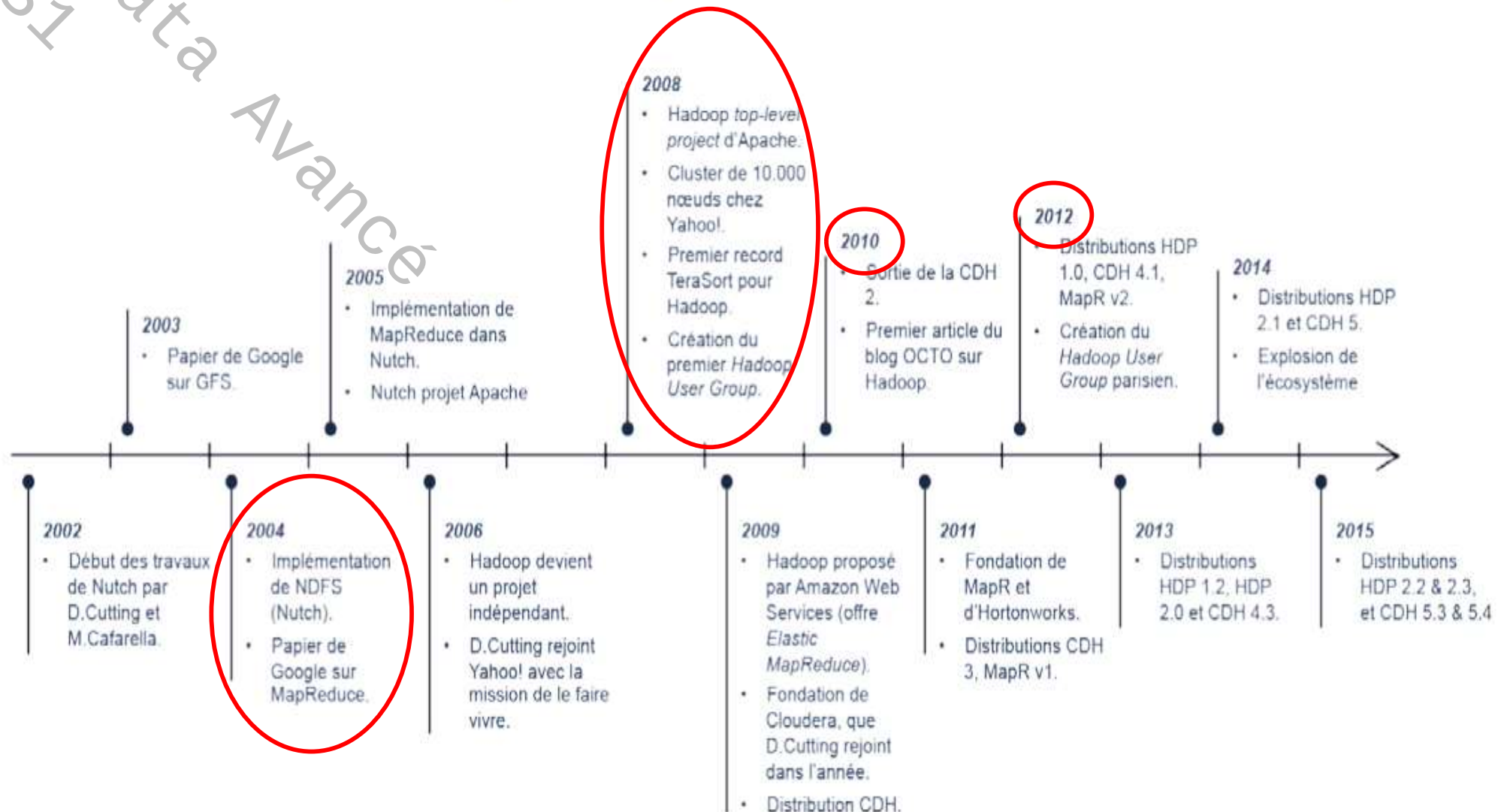


Hadoop: Main Concept



- Hadoop began as a project called **Nutch**, an open source research project on distributed systems.
- In 2003-2004, **Google** published the Google Mapreduce and GFS documents.
- Mapreduce was adapted for Nutch.
- **Doug Cutting** and **Mike Cafarella** are the creators of Hadoop.
- When Doug Cutting joined **Yahoo**, a new project was created along similar lines to Nutch: called **Hadoop**.
- Several **projects and sub-projects** began to integrate with Hadoop, called the **Hadoop ecosystem**.

Hadoop : les grandes dates clefs



- **Low cost** - Runs on commodity hardware:
Hadoop can run on basic hardware (average performance) and does not require a high-performance system. This keeps costs down. Adding or removing nodes from the cluster is simple and on-demand. The cost per terabyte is lower for storage and processing in Hadoop.
- **Storage flexibility:**
Hadoop can store data in **raw format** in a distributed environment. Hadoop can process unstructured and semi-structured data better than most available technologies.
- **Open source community:**
Hadoop is open source and supported by many contributors with a growing network of developers worldwide. **Supported** by the giants of the data industry (**Yahoo, Facebook, Hortonworks, Cloudera, ...**)

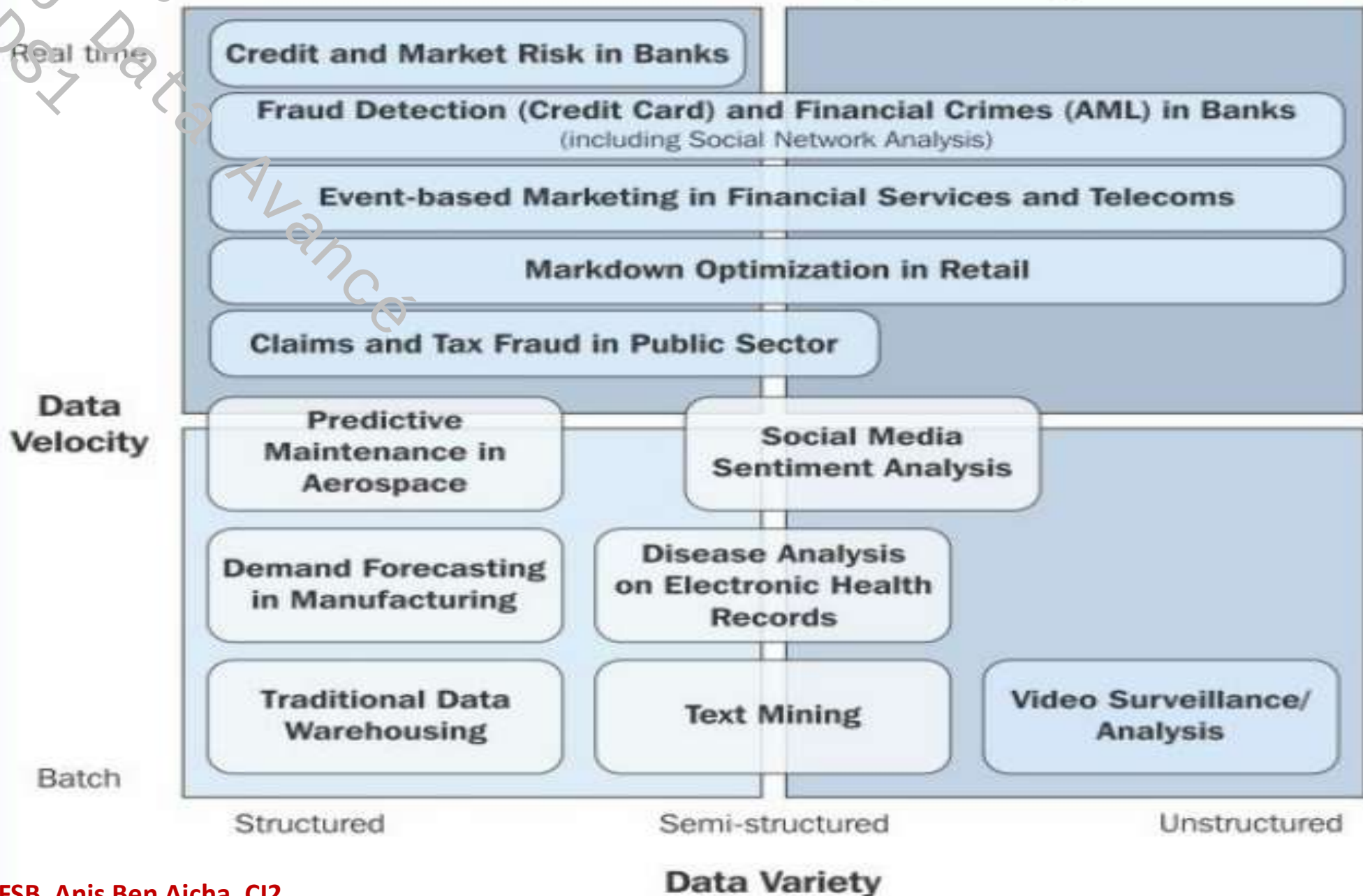
- **Fault tolerance:**

Hadoop is massively scalable and fault tolerant. Hadoop is reliable in terms of data availability, and even if some nodes are unavailable, Hadoop can recover the data. **The Hadoop architecture assumes that nodes can shut down** and the system must be able to process the data.

- **Analysis of complex data:**

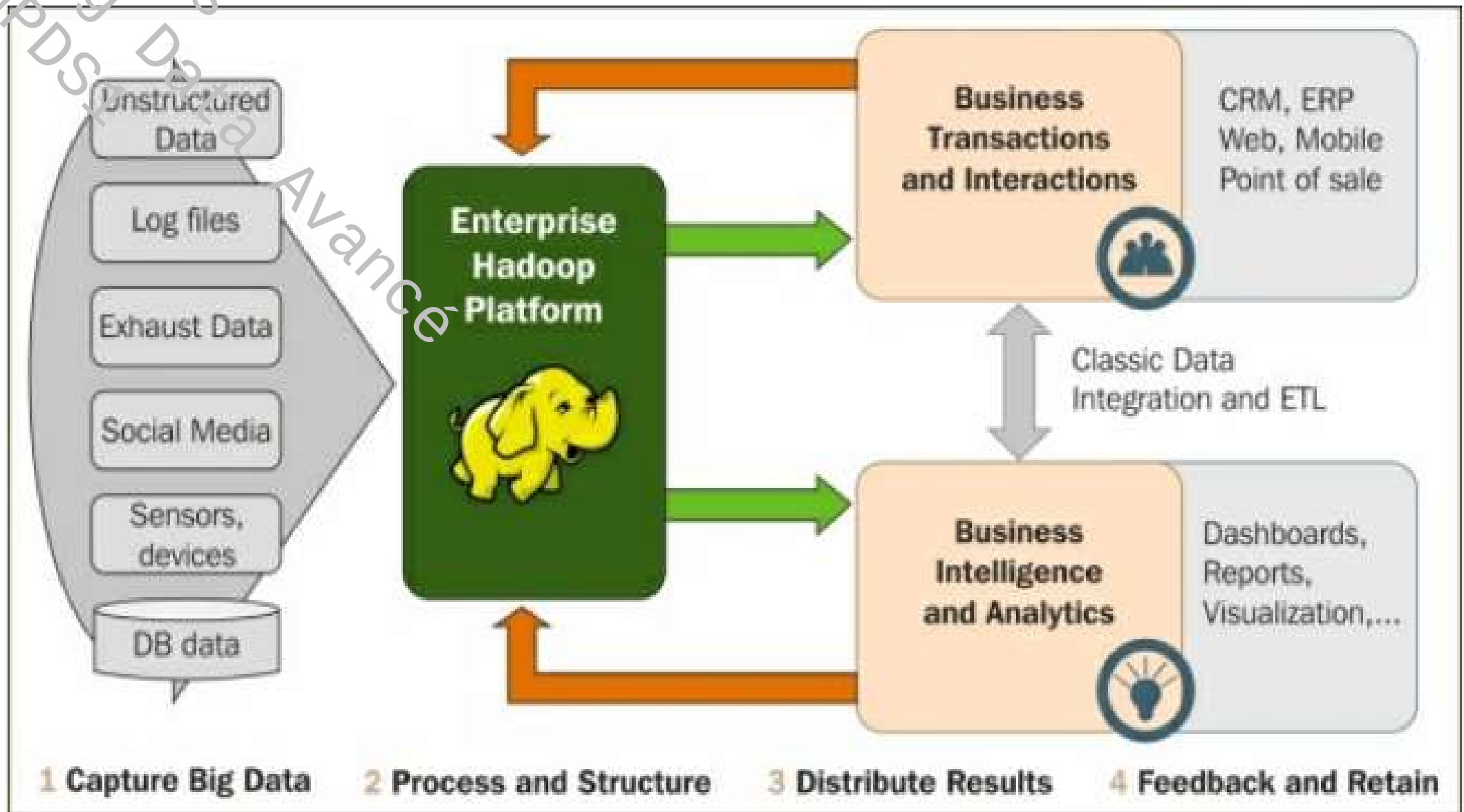
With the emergence of Big Data, data science has also made giant leaps, and complex algorithms have seen the light of day. Hadoop can handle such scalable algorithms for very large-scale data and can process the algorithms faster.

Potential Use Cases for Big Data Analytics



- **Capture Big Data** : The sources can be extensive lists that are structured, semi-structured, and unstructured, some streaming, real-time data sources, sensors, devices, machine-captured data, and many other sources. For data capturing and storage, we have different data integrators such as, **Flume**, **Sqoop**, **Storm**, and so on in the Hadoop ecosystem, depending on the type of data.
- **Process and Structure**: cleansing, filtering, and transforming the data by using a MapReduce-based framework or some other frameworks which can perform distributed programming in the Hadoop ecosystem. The frameworks available currently are **MapReduce**, **Hive**, **Pig**, **Spark** and so on.
- **Distribute Results**: The processed data can be used by the BI and analytics system or the big data analytics system for performing **analysis** or **visualization**.
- **Feedback and Retain**: The data analyzed can be fed back to Hadoop and used for improvements and audits.

Hadoop: Basic Data Flow



- Hadoop architecture is designed to be easily integrated with other systems.
- Integration is very important because although we can process the data efficiently in Hadoop, **but we should also be able to send that result to another system** to move the data to another level. Data has to be integrated with other systems to achieve interoperability and flexibility.
- Some implemented tools
 - **Data Integration tools** such as, **Sqoop, Flume**, and others
 - **NoSQL tools** such as, **Cassandra, MongoDB, Couchbase**, and others
 - **ETL tools** such as, **Pentaho, Informatica, Talend**, and others
 - **Visualization tools** such as, **Tableau, Power BI, Sas, R**, and others

BI/Analytics Tools



Karmasphere

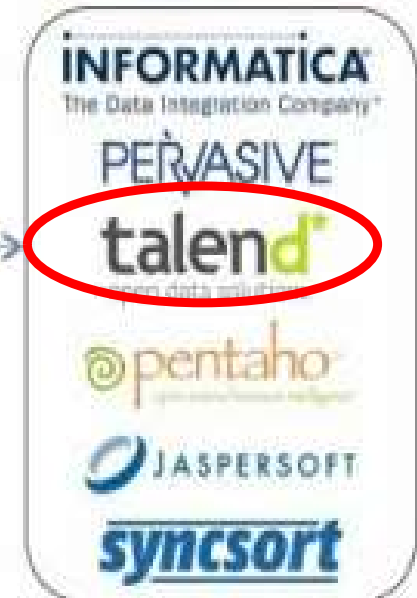
MicroStrategy



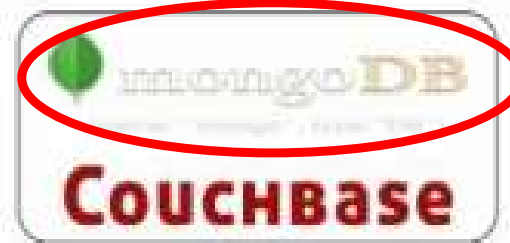
REVOLUTION
ANALYTICS



Data Import/Export



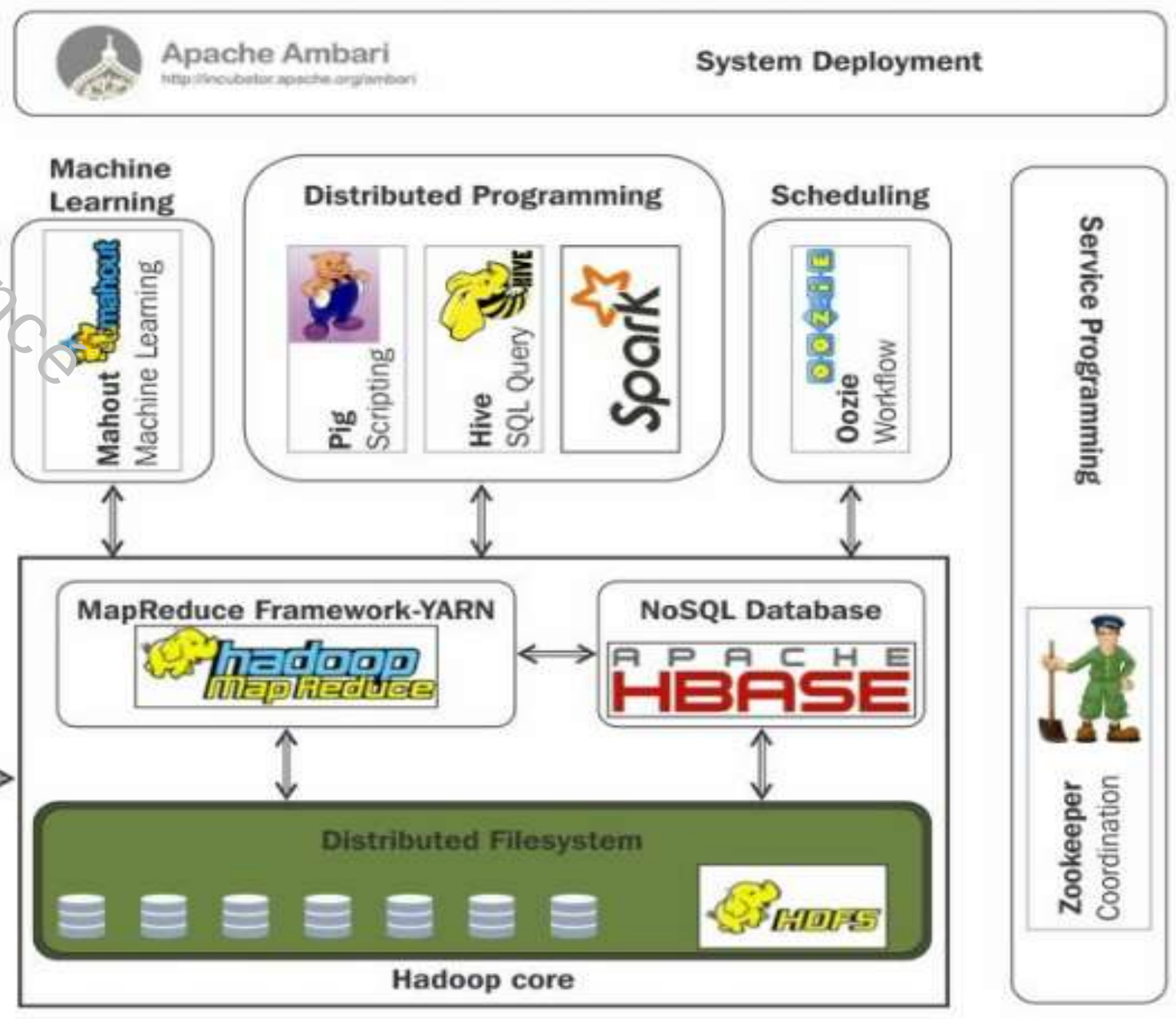
Data Integration Tools



NoSQL

- Hadoop ecosystem comprises of a lot of sub-projects
- Subproject can be configured as we need in a Hadoop cluster.
- As Hadoop is an open source software and has become popular, a lot of contributions and improvements supporting Hadoop by different organizations have been made.

Hadoop: Ecosystem



▪ 1) Distributed filesystem

In Hadoop, data is stored in a **distributed computing** environment, so the files are scattered across the cluster. We should have an efficient filesystem to manage the files in Hadoop. The filesystem used in Hadoop is HDFS, elaborated as Hadoop Distributed File System.

- HDFS

HDFS is extremely scalable and fault tolerant. It is designed to efficiently process parallel processing in a distributed environment in even commodity hardware. HDFS has daemon processes in Hadoop, which manage the data. The processes are **NameNode, DataNode, BackupNode, and Checkpoint NameNode**.

■ 2) Distributed programming

- To leverage the power of a distributed storage filesystem, Hadoop performs distributed programming which can do massive parallel programming.
- Distributed programming is the heart of any big data system, so it is extremely critical.
- Different frameworks that can be used for **distributed programming**:
 - MapReduce
 - Hive
 - Pig
 - Spark
- The basic layer in Hadoop for distributed programming is MapReduce.

■ Hadoop MapReduce:

- MapReduce is the **heart of the Hadoop** system distributed programming.
- MapReduce is a framework model designed as **parallel processing** on a distributed environment.
- Hadoop MapReduce was inspired by Google MapReduce whitepaper.
- Hadoop MapReduce is scalable and massively parallel processing framework, which can work on huge data and is designed to run, even in commodity hardware.
- Before Hadoop **2.x**, MapReduce was the **only processing** framework that could be performed.

■ Apache Hive:

- Hive provides a **data warehouse infrastructure** system for Hadoop, which creates a **SQL-like wrapper** interface called **HiveQL**, on top of MapReduce.
- Hive can be used to run some ad hoc querying and basic aggregation and summarization processing on the Hadoop data.
- Hive is designed on top of MapReduce, which means a **HiveQL query will run the MapReduce jobs** for processing the query.

■ Apache Pig:

- Pig provides a **scripting-like wrapper** written in the **Pig Latin** language to process the data with script-like syntax.
- Pig was developed by Yahoo and contributed to Apache.
- Pig also **translates the Pig Latin** script code **to MapReduce** and executes the job.
- Pig is usually used for analyzing semi-structured and large data sets.

■ Apache Spark:

- Spark provides a powerful **alternative** to Hadoop's MapReduce.
- Apache Spark is a parallel data processing framework that can run programs **up to 100 times faster than Hadoop MapReduce** in memory, or 10 times faster on disk.
- Spark is used for **real-time stream processing** and analysis of the data.

■ 3) NoSQL databases

- Within Hadoop ecosystem, we have a **NoSQL database** called HBase.
- HBase is one of the key component that provides a very flexible design and **high volume simultaneous reads and write** in **low latency** hence it is widely adopted.
- HBase is a NoSQL, **column oriented database** and a key/value store, which works on top of HDFS.
- HBase provides **faster lookup** and also high volume inserts/updates of a random access request on a high scale.
- The HBase **schema is very flexible** and actually variable, where the columns can be added or removed at runtime.
- HBase supports low-latency and strongly consistent read and write operations. It is suitable for high-speed counter aggregation.

■ 4) Data ingestion

- **Data management** in big data **is an important and critical aspect**. We have to import and export large scale data to do processing, which becomes unmanageable in the production environment.
- In Hadoop, **we deal with different set of sources** such as batch, streaming, real time, and also sources that are complex in data formats, as some are semi-structured and unstructured too. Managing such data is very difficult, therefore we have some **tools for data management** such as **Flume, Sqoop, and Storm**

■ Apache Flume:

- Apache Flume is a widely used tool for **efficiently collecting, aggregating, and moving** large amounts of log data from many different sources to a **centralized data store**. Flume is a distributed, reliable, and available system. It **performs well if a source is streaming**, for example, log files.

■ Apache Sqoop:

- Sqoop can be used to manage data between **Hadoop** and **relational databases**, enterprise **data warehouses**, and **NoSQL systems**.
- Sqoop can import and export data in MapReduce, and can import and export data in parallel mode.
- Sqoop is also fault tolerant.

■ Apache Storm:

- Apache Storm provides a **real-time, scalable, and distributed** solution for streaming data.
- Apache Storm can be used with any programming language and it guarantees that data streams are processed without data loss.
- Storm is **datatype-agnostic**, it processes data streams of **any data type**.

■ 5) Service programming

- Programming in a distributed environment is complex and care has to be taken, otherwise it can become inefficient.
- To develop properly distributed applications in Hadoop, we have some service programming tools which provide utilities that take care of the distribution and resource management aspect.
 - Apache YARN
 - Apache Zookeeper

6) Scheduling

- The Hadoop system **can have multiple jobs** and these **have to be scheduled** many times.
- Hadoop jobs' scheduling is complex and difficult to **create, manage, and monitor**.
- We can use a system such as **Oozie** to coordinate and monitor Hadoop jobs efficiently
- **Apache Oozie**:
 - Oozie is a **workflow and coordination** service processing system that lets the users manage multiple jobs as well as chain of jobs written in MapReduce, Pig, and Hive, also java programs and shell scripts too, and can link them to one another.
 - Oozie can be used to set rules for beginning and ending a workflow and it can also detect the completion of tasks.

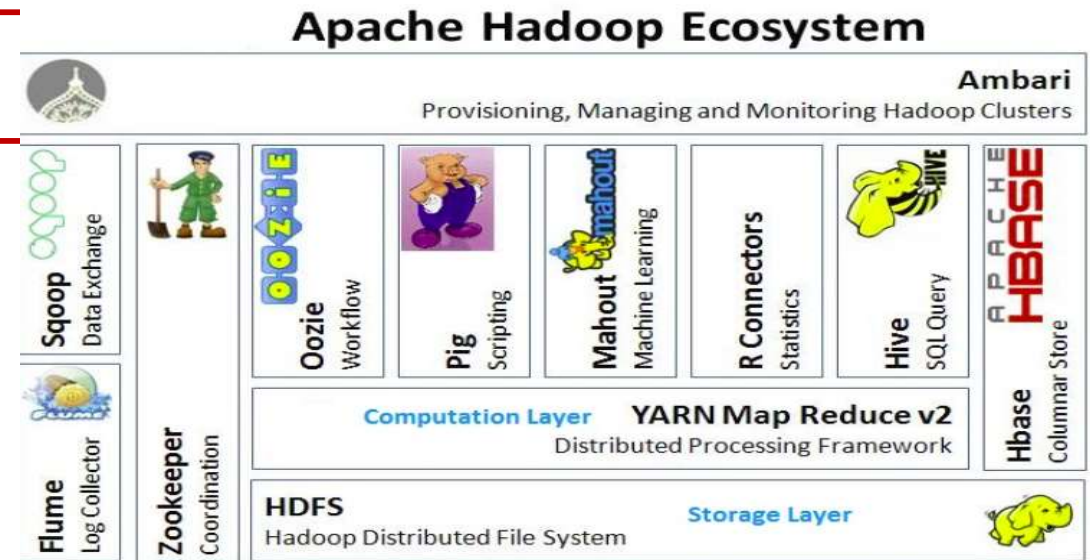
■ 7) Data analytics and machine learning

- In Hadoop, and for general big data, **analytics is the key interest area**, as Hadoop is a powerful tool to process complex programs and algorithms to improve the process and business.
- Due to the powerful processing nature of Hadoop, **machine learning** has been in focus and a lot of development in the algorithms and techniques have been adapted for Hadoop.
- **Apache Mahout:**
 - Mahout is a scalable machine learning API, which has a lot of implemented **machine learning libraries**.
 - Mahout is an isolated project which can be used as a pure machine learning library, but the power of Mahout enhances when it is integrated with Hadoop.
 - Some of the algorithms which are popularly used in Mahout
 - Recommendation
 - Clustering
 - Classification

■ 8) System management

- **Deploying, provisioning, managing, and monitoring** a Hadoop cluster requires expert scripting knowledge and usually takes a good amount of effort and time manually, but is repetitive. For performing such activities in Hadoop, we can use tools such as Ambari.
- Apache Ambari
 - Ambari can be used by application developers and system integrators for **managing most of the administration activities** in a Hadoop cluster.
 - Ambari is an open source framework in the Hadoop ecosystem, which can be used for **installing, provisioning, deployment, managing, and monitoring** a Hadoop cluster.
 - Ambari's main motive is **to hide the complexity** of the Hadoop

HDFS: Architecture



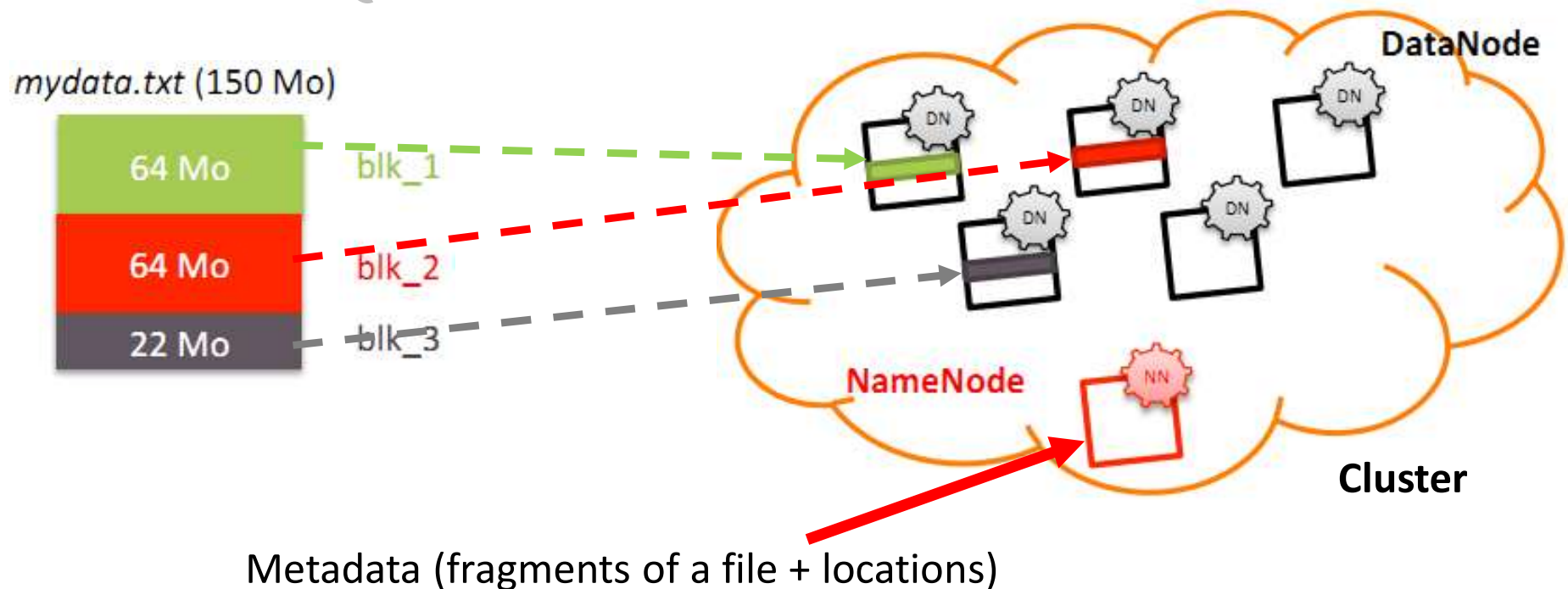
- HDFS is a **distributed, extensible** and **portable** file system
- Written in Java
- Allows **very large volumes** of data to be stored on a **large number of machines** (nodes) equipped with standard cluster hard disks
- Ex. When a mydata.txt file is saved in HDFS, it is broken down into large blocks (64MB < Hadoop 2.2.0 & 128MB from Hadoop 2.2.0), each block having a unique name: blk_1, blk_2...



HDFS: Architecture



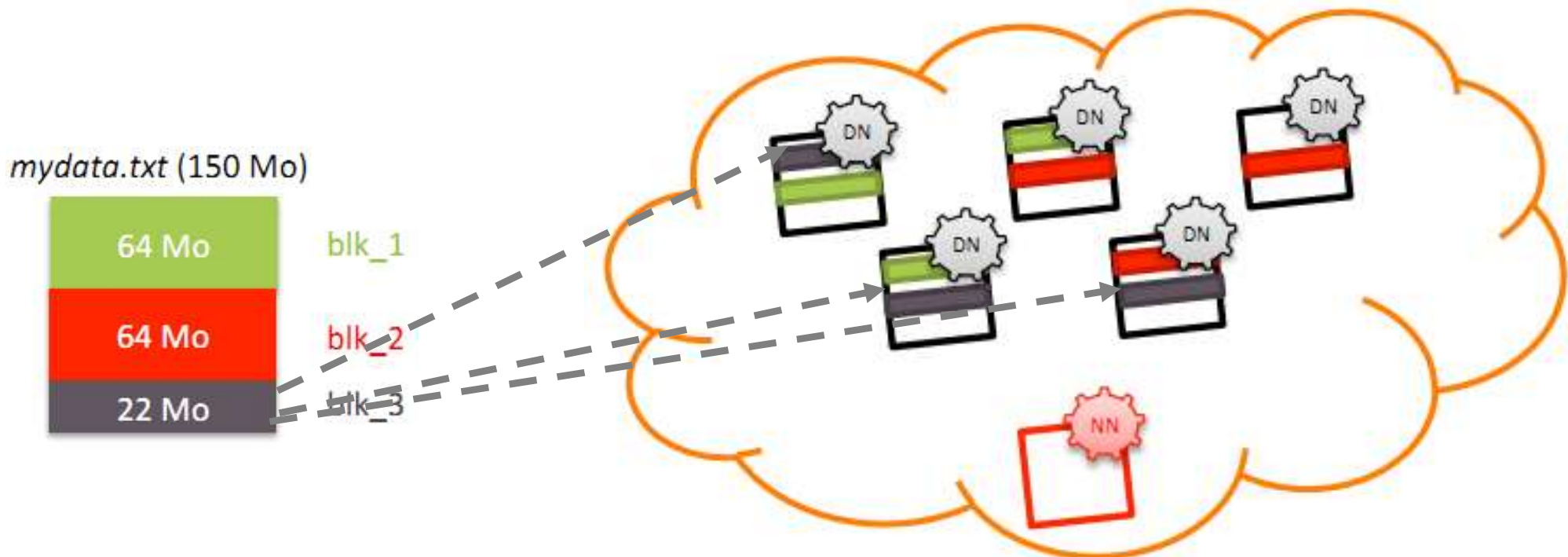
- Each block is stored on a different node in the cluster
- DataNode**: daemon on each node in the cluster
- NameNode** : daemon running on a separate machine
 - Contains metadata
 - Allows to find the nodes running the blocks in a file



HDFS: Architecture



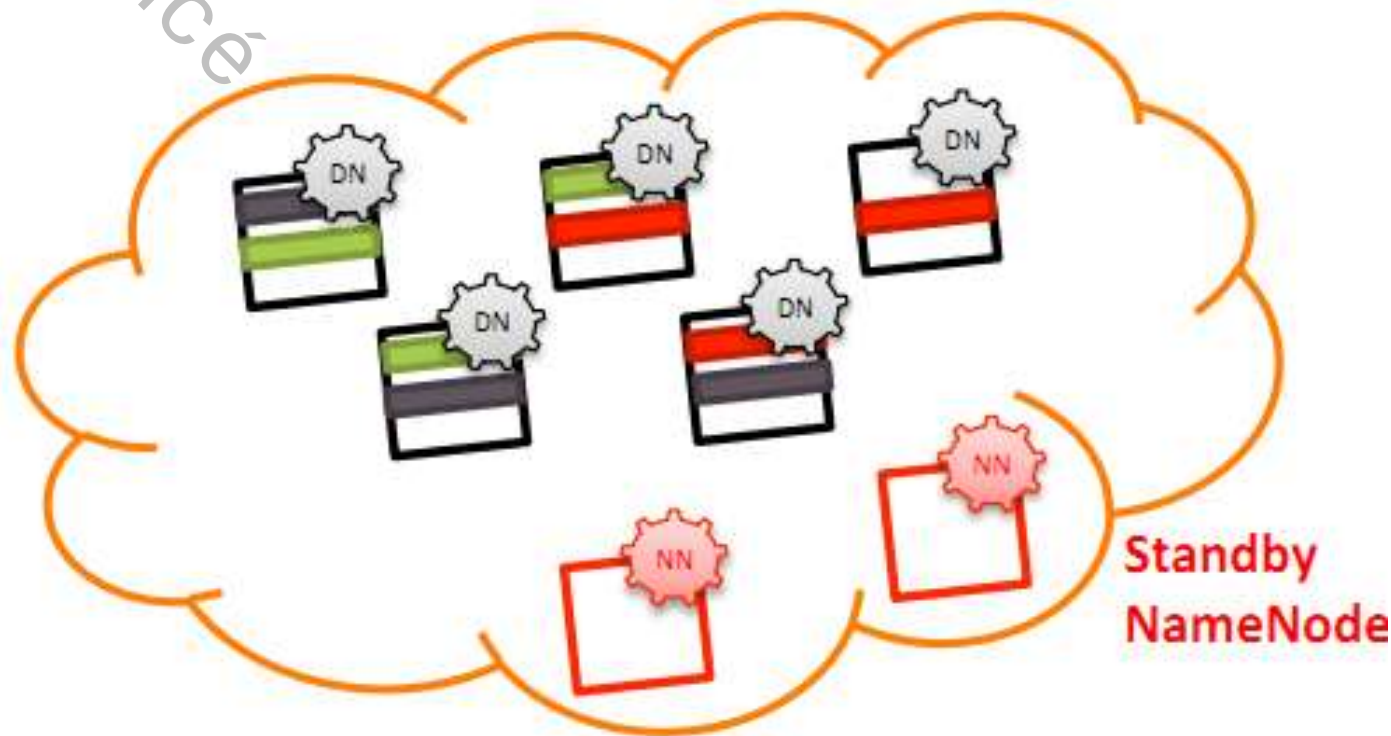
- If one of the nodes has a problem, the data will be lost
 - Hadoop replicates each block 3 times (by default)
 - It chooses 3 nodes at random, and places a copy of the block in each of them
 - If the node is down, the NN detects it, and replicates the blocks that were hosted there so that 3 copies are always stored.
 - Rack Awareness concept



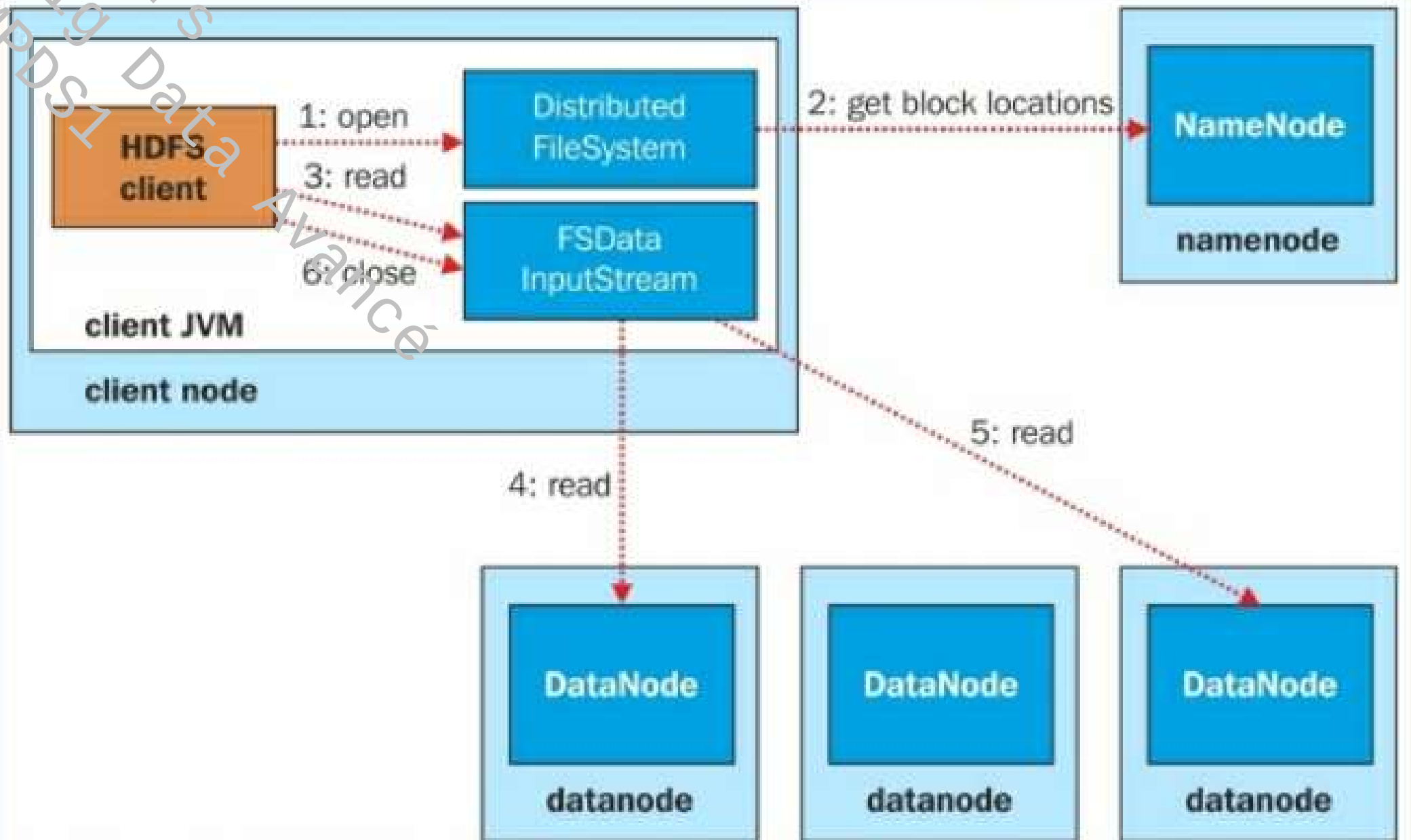
HDFS: Architecture



- If the NameNode has a problem :
 - Data lost forever
 - Data inaccessible
- Define another NN (**standby NameNode**) to resume work if the active NameNode fails

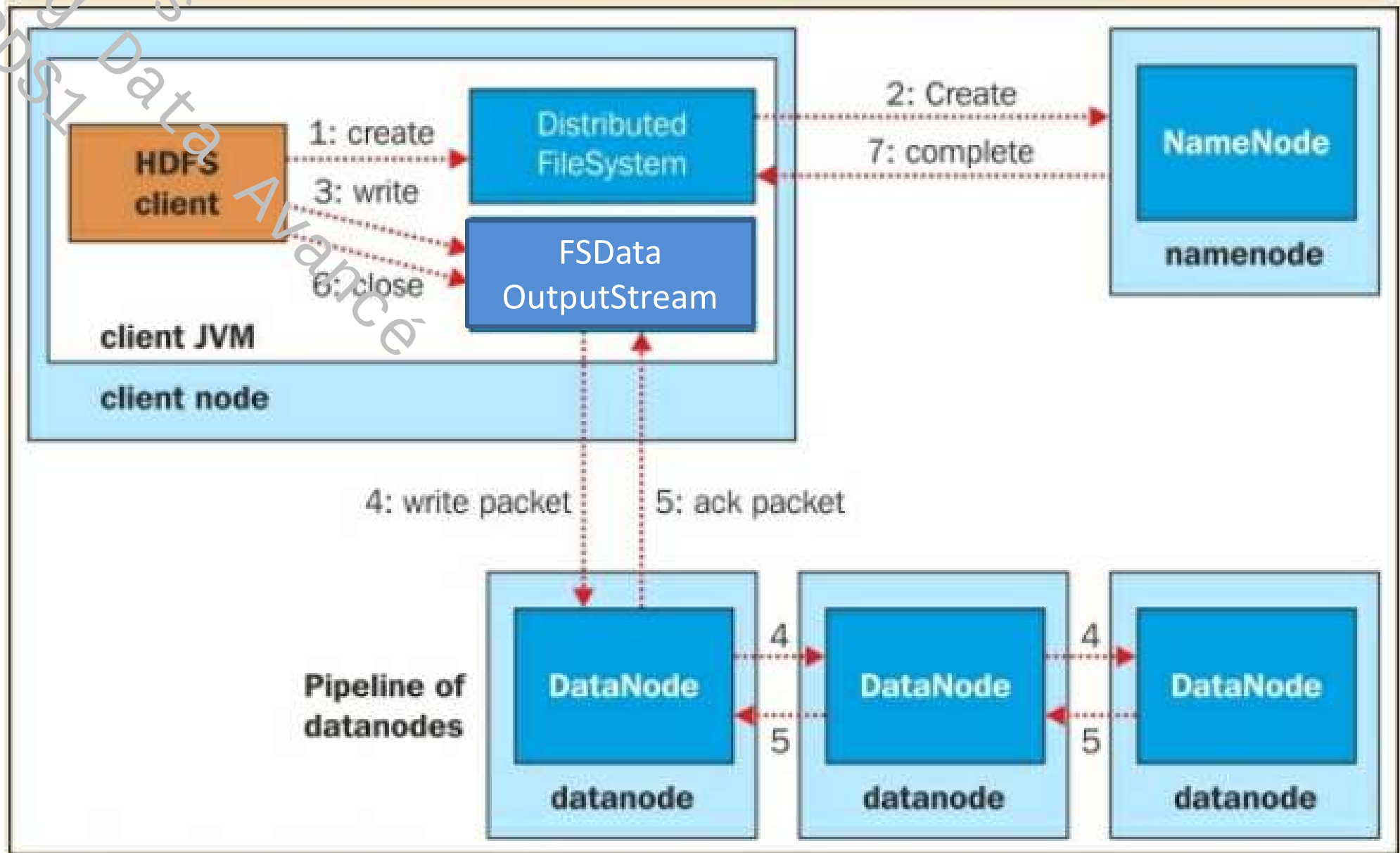


HDFS: Read/Write



- The HDFS **read process** involves the following **six steps**:
 - 1) The client using a Distributed FileSystem object of Hadoop client API calls **open()** which **initiate the read request**.
 - 2) Distributed FileSystem connects with NameNode. NameNode identifies the block locations of the file to be read and in which DataNodes the block is located. NameNode then sends the list of DataNodes in order of nearest DataNodes from the client.
 - 3) Distributed FileSystem then creates FSDataInputStream objects, which, in turn, wrap a DFSInputStream, which can connect to the DataNodes selected and get the block, and return to the client. The client initiates the transfer by calling the read() of FSDataInputStream.
 - 4) FSDataInputStream repeatedly calls the read() method to get the block data.
 - 5) When the end of the block is reached, DFSInputStream closes the connection from the DataNode and identifies the best DataNode for the next block.
 - 6) When the client has finished reading, it will call close() on FSDataInputStream to close the connection.

HDFS: Read/Write



Write pipeline

The HDFS write pipeline process flow is described in **seven steps**:

- 1) The client, using a Distributed FileSystem object of Hadoop client API, calls **create()**, which **initiates the write request**.
- 2) Distributed FileSystem connects with NameNode. NameNode initiates a new file creation, and creates a new record in metadata and initiates an output stream of type FSDataOutputStream, which wraps DFSOutputStream and returns it to the client. Before initiating the file creation, NameNode **checks if a file already exists** and whether the client **has permissions to create** a new file and if any of the condition is true then an IOException is thrown to the client.
- 3) The client uses the FSDataOutputStream object to write the data and calls the **write()** method. The FSDataOutputStream object, which is DFSOutputStream, handles the communication with the DataNodes and NameNode.

4) DFSOutputStream splits files to blocks and coordinates with NameNode to identify the DataNode and the replica DataNodes. The number of the replication factor will be the number of DataNodes identified. Data will be sent to a DataNode in packets, and that DataNode will send the same packet to the second DataNode, the second DataNode will send it to the third, and so on, until the number of DataNodes is identified.

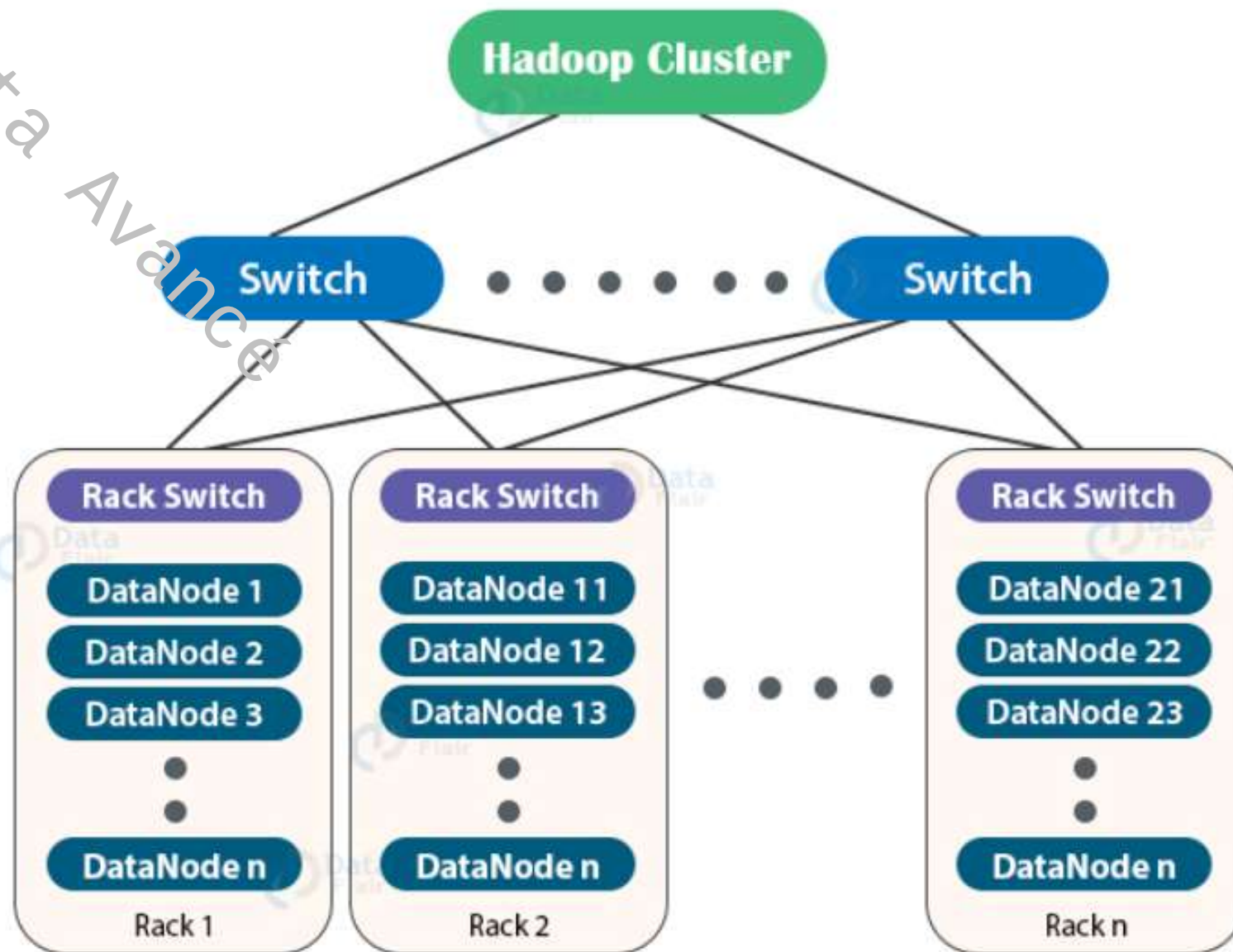
5) When all the packets are received and written, DataNodes send an acknowledgement packet to the sender DataNode, to the client.

DFSOutputStream maintains a queue internally to check if the packets are successfully written by DataNode. DFSOutputStream also handles if the acknowledgment is not received or DataNode fails while writing.

6) If all the packets have been successfully written, then the client closes the stream.

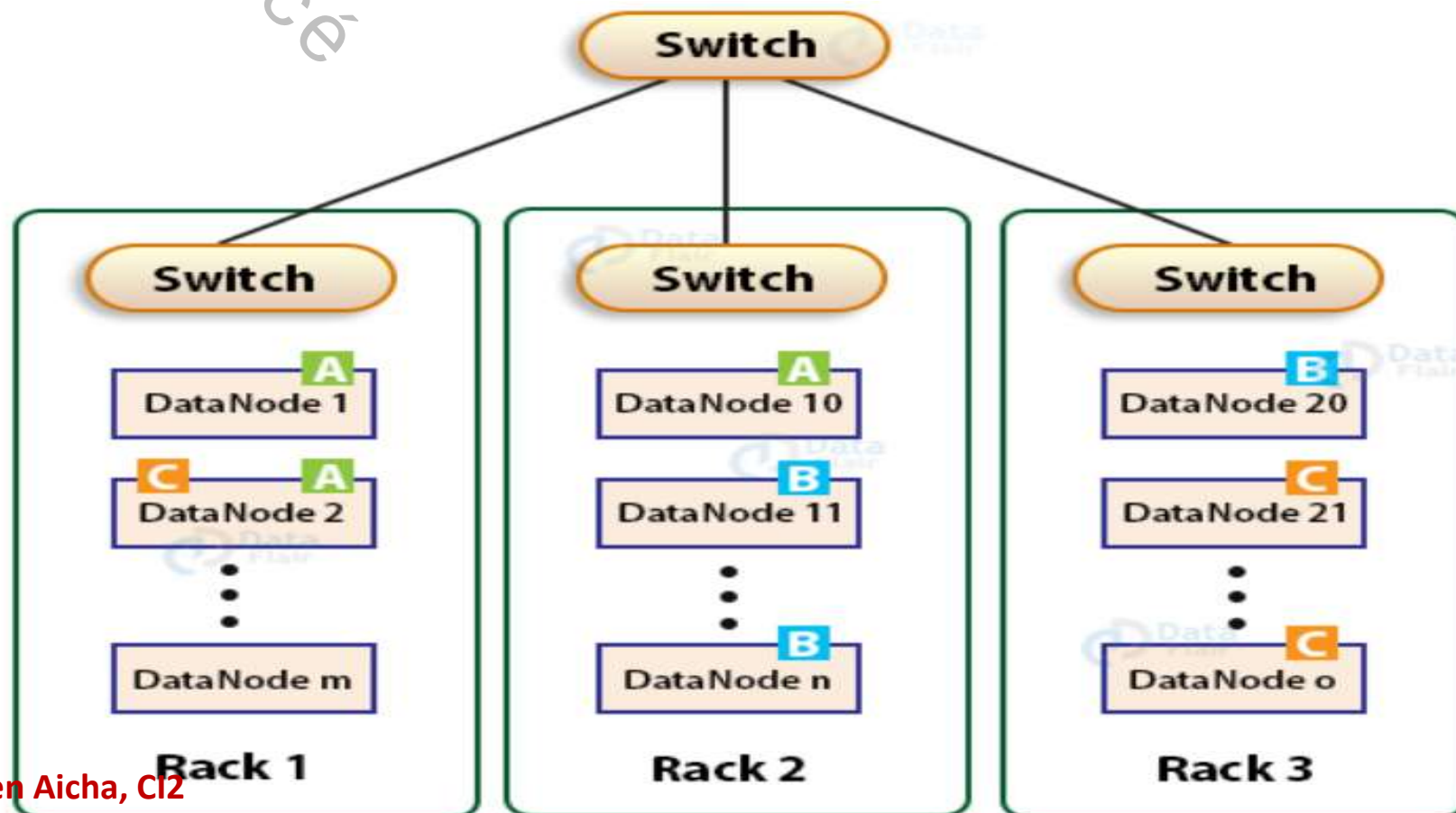
7) If the process is completed, then the Distributed FileSystem object notifies the NameNode of the status.

- HDFS stores files on multiple nodes (DataNodes) in a cluster. To get the maximum performance from Hadoop and to improve network traffic during file reads/writes, NameNode chooses DataNodes on the same rack or nearby racks for the read/write data. "Rack awareness is the concept of choosing the nearest DataNode based on rack information".
- The Rack is the collection of approximately 40-50 DataNodes connected using the same network switch. If the network fails, the whole rack will be unavailable. A large Hadoop cluster is deployed in several racks.



- In a large Hadoop cluster, there are several racks. Communication between DataNodes in the same rack is more efficient than communication between DataNodes in different racks.
- **Advantages of Rack Awareness**
 - Reduce network traffic during file read/write, which improves cluster performance.
 - To achieve fault tolerance, even when the rack is no longer accessible
 - To achieve high data availability even under adverse conditions.
 - To reduce latency (i.e. ensure that the file is read/written with as little delay as possible).

- NameNode on multiple rack clusters maintains block replication using the following **Rack Awareness policies**:
 - No more than one replica is placed on a node.
 - No more than two replicas are placed on the same rack.
 - In addition, the number of racks used for block replication should always be smaller than the number of replicas.



- The basic HDFS architecture supports a single NameNode that manages the file system name space.
 - **Memory** becomes the **limiting** factor for scale
 - **Bottleneck** problem.
- The HDFS Federation feature added in Hadoop 2.0 allowed a cluster to scale by adding more NameNodes.

HDFS Basic

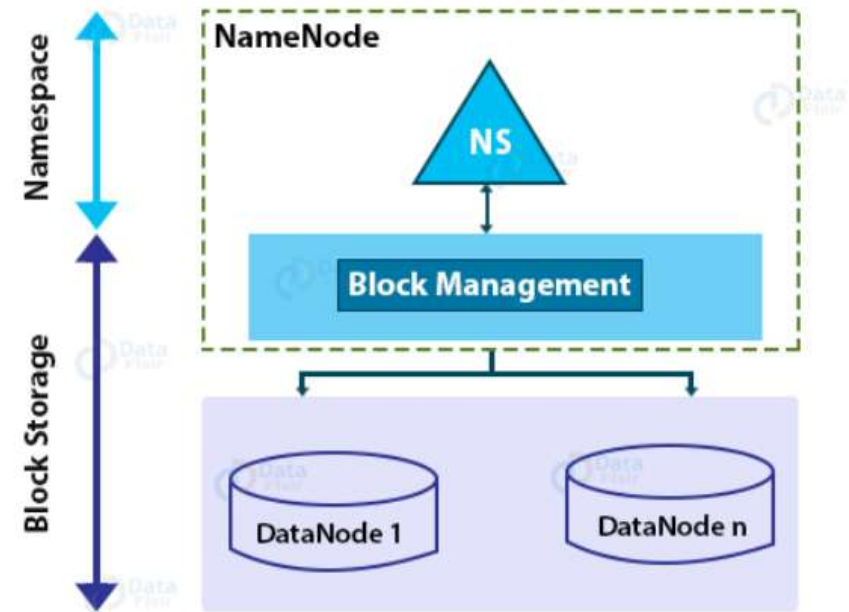
■ Namespace

The Namespace layer of the HDFS architecture **consists of files, blocks and directories**. This layer provides support for namespace-related file system operations such as creating, deleting, modifying and listing files and directories.

■ **The block storage layer** has two parts:

- **Block Management:** **NameNode performs block management**. Block Management provides DataNode membership of the cluster by **handling periodic registrations and beats**. It processes **block reports** and supports block-related operations such as **create, delete, modify or get block location**. It also maintains block location and replica placement.
- **Storage:** DataNode manages storage space by storing blocks on the local file system.

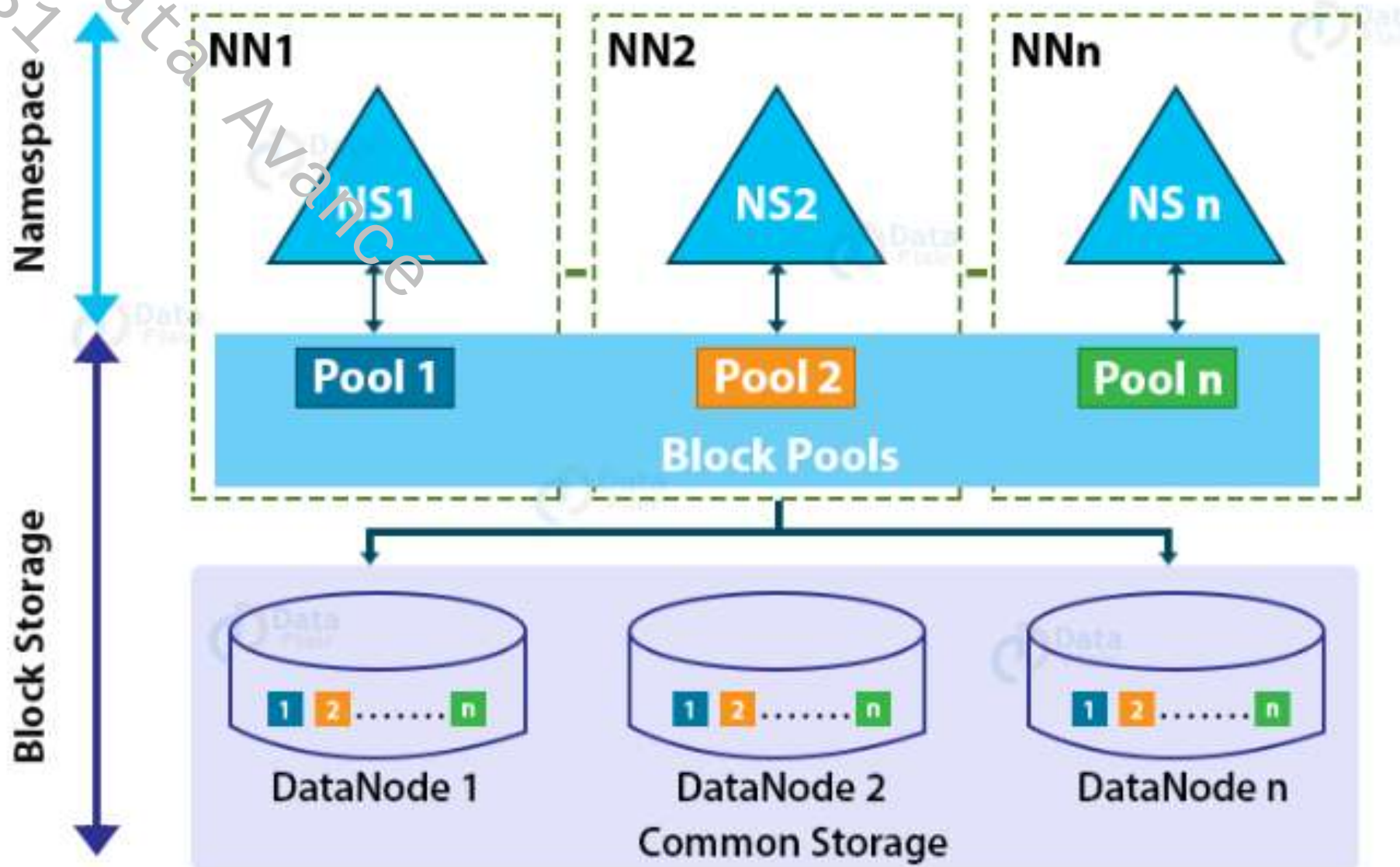
HDFS Layers



- The basic architecture only allows a single NameNode to maintain the file system name space.
- Simple to implement and works well for small clusters.
- Not suitable for large organizations (Yahoo, Facebook, etc.)
- **Limitations of the current HDFS architecture**
 - Thanks to the single NameNode, we can only have a **limited number of DataNodes** that a single NameNode can manage.
 - Filesystem operations are also **limited to the number of jobs that NameNode manages at any one time**. As a result, cluster performance depends on NameNode throughput.
 - Because of a single namespace, there is **no isolation** among the occupant organizations using the cluster.

- The HDFS Federation feature introduced in **Hadoop 2** enhances the existing HDFS architecture.
- It overcomes the limitations of the HDFS architecture by **adding multiple NameNode / namespaces**.
- As a result → horizontal scalability.
- In the HDFS Federation architecture, **there are several NameNodes** and DataNodes.
- **Each NameNode** has its own namespace and **block pool**.
- **All NameNodes use all DataNodes as common storage**.
- Each NameNode is independent of the other (no co-ordination is required between them).
- **Each Datanode is registered to all NameNodes** in the cluster
- DataNodes periodically send heartbeats to all NameNodes

HDFS Federation Architecture



Commande en ligne
API JAVA
HTTP
FTP

- There are 2 possible syntaxes:
 - With hadoop: syntax = **hadoop fs <command>**.
 - With hdfs: syntax = **hdfs dfs <command>**.
- These commands are similar to those used by the Linux shell, such as ls, mkdir, rm and cat.
- Two types of command
 - Linux Like :
cat, chgrp, chmod, chown, cp, du, ls, mkdir, mv, rm, stat, tail...
 - Specific commands
copyFromLocal, put, copyToLocal, get, getmerge, setrep...
- Note
The command **hadoop fs -help** displays the list of HDFS commands

HDFS:

Commands



To list the contents of a directory

hdfs dfs -ls <chemin du répertoire>

For example:

hdfs dfs -ls /

hdfs dfs -ls /user # pour voir le contenu du répertoire "user"

To display the contents of a file

hdfs dfs -cat <chemin du fichier>

For example:

hdfs dfs -cat /user/myFile.txt

HDFS: Commands



Create directory

hdfs dfs -mkdir <chemin du nouveau répertoire>

Par exemple:

hdfs dfs -mkdir /user/output

Copy a file in HDFS:

hdfs dfs -put <chemin du fichier source> <chemin du fichier destination sur HDFS>

The following command is reserved for local files only:

hdfs dfs -copyFromLocal <chemin du fichier sr> <chemin du fichier dest sur HDFS>

For example:

hdfs dfs -put ~/TextFile.txt /user

or

hdfs dfs -copyFromLocal ~/TextFile.txt /user

To retrieve a file from HDFS

hdfs dfs -get <chemin du fichier sur HDFS> <chemin du fichier en local>

For example:

hdfs dfs -get /user/TextFile2.txt ~/copyFromHadoop

The following command is reserved for local files only:

hdfs dfs -copyToLocal /user/TextFile2.txt ~/copyFromHadoop

HDFS:

Commands

To empty the recycle bin

```
hadoop fs -expunge
```



the **setrep** command changes the replication factor to a specific account instead of the default replication factor. If used for a directory, it will recursively change the replication factor for all files residing in the directory.

```
hadoop fs -setrep 5 /user
```

To check the file size

```
hadoop fs -du /user/myFile.txt
```

The **df** command shows the capacity, size and free space available on the HDFS file system.

```
hadoop fs -df
```

HDFS:

Commands

To test whether a file exists or is empty (1 if yes
0 if no)

hadoop fs -test /user/myFile.txt

Help on using a particular command

hadoop fs -usage <command>

To change the permissions of a file

hadoop fs -chmod -r /user/myFile.txt

File append

hadoop fs -appendToFile ~/file1 ~/file2 /user/appendFile

touchz 01

07 chmod

test 02

08 appendToFile

text 03

09 checksum

stat 04

10 count

usage 05

11 find

help 06

12 getmerge

- Hadoop offers a complete Java API for accessing HDFS fichiers. It is based on two main classes:
 - **FileSystem** represents the tree of files (file system). This class can be used to copy local files to HDFS (and vice versa), rename, create and delete files and folders.
 - **FileStatus** manages information about a files or folder:
 - size with `getLen()` ,
 - nature with `isDirectory()` and `isFile()` ,
- These two classes need to know the configuration of the HDFS cluster, using the Configuration class. In addition, the full names of the files are represented by the Path class.

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.FileStatus;
import org.apache.hadoop.fs.Path;
import java.io.*;

public class HDFSread {
    public static void main (String[] args) throws IOException {
        Configuration conf = new Configuration();
        FileSystem fs = FileSystem.get(conf);
        Path nomcomplet = new Path( "myFile.txt" );
        FSDataInputStream inStream = fs.open(nomcomplet);
        InputStreamReader isr = new InputStreamReader(inStream);
        BufferedReader br = new BufferedReader(isr);
        String line = br.readLine();
        System.out.println(line);
        inStream.close();
        fs.close ();
    }
}
```

- 1) Create a "rep1" Hadoop HDFS directory directly on the root
- 2) You have a "myFile.txt" file in local ~/MPDS/Level1. Copy the myFile file to the "rep1" directory using two methods
- 3) You are asked to move the file "myFile.txt" to a second directory "rep2".
- 4) Display the header and the tail of the myFile file from "rep2".
- 5) Check its size
- 6) Make sure that the file is duplicated 4 times on the Hadoop cluster. Taking into account the federation of Hadoop, make an explanatory diagram showing how the file is stored on the cluster.
- 7) Test whether the myFile still exists in rep1

Annexe : Hadoop Cluster



Cluster of machines running Hadoop at Yahoo! (Source: Yahoo!)